

Capacity Estimation, Clustering and Load Balancing

Short study guide: concepts, business examples and Q&A

Big picture

Capacity estimation decides *how much* you need. **Clustering** makes many servers work as one system. **Load balancing** decides *which server* gets each request. Together they give scale, availability and sensible cost.

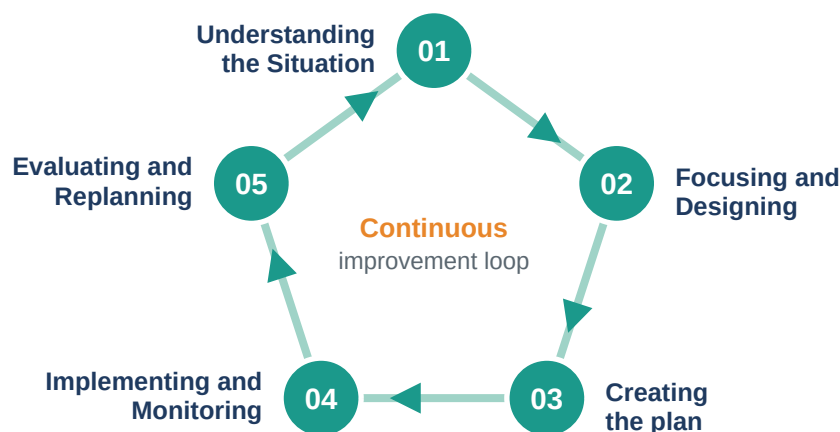
1. Capacity Estimation

Predicting the resources (CPU, memory, disk, servers) a system needs to meet performance and availability goals, now and as it grows.

Metric	Meaning	Note
Throughput	Requests or transactions per second	Measure at peak, not average
Latency	Time to complete one request	Watch p95 and p99
Response time	Total time the user waits	Latency plus queueing
Load and concurrency	Number of simultaneous users or operations	Drives threads, memory, connections
Utilization	Percentage of capacity in use	Target about 60-75% at peak
Business impact	Cost of over-provisioning vs risk of under-provisioning	Turns numbers into budget

Little's Law: concurrency = throughput x response time. Example: 500 RPS x 0.2 s = 100 concurrent requests. **Why headroom matters:** response time is roughly service time / (1 - utilization), so a 100 ms request takes about 200 ms at 50% load but about 1,000 ms at 90%.

The lifecycle (slide 56) is a loop



Six steps (slide 57)

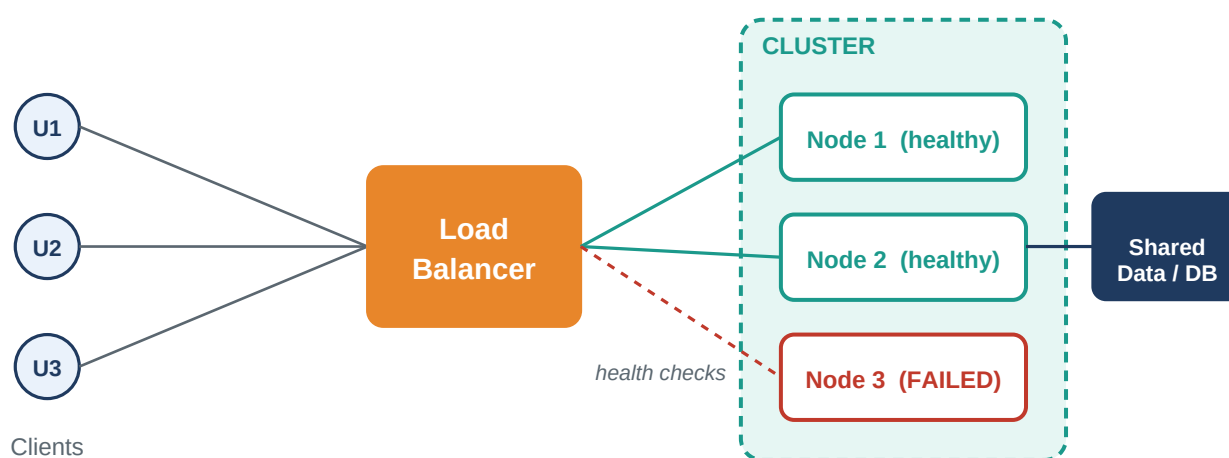
Step	What to do
1. Define requirements	Expected workload, peak traffic, availability needs
2. Analyze historical data	Find patterns, seasonality and trends
3. Model the system	Workload modelling (read vs write heavy), resource consumption (CPU, memory, disk I/O), concurrency and scaling factors
4. Conduct load testing	Stress and load tests to validate the model and find bottlenecks
5. Provision resources	Required resources for projected capacity, with a margin for peak
6. Estimate growth	Forecast workload growth from business plans

Worked example

2M daily users x 20 requests = 40M/day, about 463 RPS average, about 2,315 RPS at 5x peak. One server handles 400 RPS but is capped at 70% (280 RPS). $2,315 / 280 = 8.3$, so 9 servers, plus 1 spare = **10 servers**. With 50% growth: $3,473 / 280 = 12.4$, so $13 + 1 = 14$ servers.

2. Clustering and Load Balancing

A **cluster** is a group of nodes acting as one logical system. A **load balancer** sits in front and spreads requests across them, removing failed nodes via health checks.



	Clustering	Load balancing
Gives you	High availability, scalability, fault tolerance, resource management	Efficient utilization, better performance, redundancy, scalability
Focus	Nodes cooperating, sharing state, failing over	Routing traffic to healthy servers
Tools	Kubernetes, Redis Cluster, Galera	NGINX, HAProxy, AWS ALB
Together	The cluster is the pool; the balancer feeds it	The balancer fronts the cluster

Algorithms: round robin (equal turns), weighted round robin (stronger servers get more), least connections (best for uneven or long requests), IP/consistent hash (sticky routing). **Layer 4** routes on IP and port (fast); **Layer 7** routes on HTTP details such as path and headers (flexible). **Design rules:** keep servers stateless, keep spare capacity (N+1), and run the balancer redundantly so it is not a single point of

failure.

3. Business Examples

Business	How it applies
E-commerce flash sale	Estimate peak from last year's campaign logs, load test checkout, put separate clusters behind a layer 7 balancer, pre-warm auto-scaling. Under-provisioning means a crashed checkout in the best sales hour.
Mobile payments	Active-active nodes across data centers, redundant balancers, big headroom for salary-day peaks. Downtime means failed payments and lost trust.
Video streaming	Route users to the nearest region, run each service as a cluster, add capacity before big releases. Buffering drives customers away.
Exam result or ticket portal	Everyone arrives at one moment: CDN caching, read replicas, rate limiting and a waiting-room queue.

4. Technical Q&A

Q1. Latency vs response time?

Latency is the time to complete the request. Response time is what the user waits in total, including queueing and network. Under heavy load response time grows far faster than latency.

Q2. A service handles 800 RPS at 250 ms. How many concurrent requests?

Little's Law: $800 \times 0.25 = 200$ concurrent requests. Size pools above this with headroom.

Q3. Why not run servers at 95% utilization?

Queueing delay explodes near saturation (about 20x service time at 95%) and there is no room for bursts or failover.

Q4. Clustering vs load balancing?

Clustering groups servers to work together with shared coordination and failover. Load balancing distributes incoming requests. They are normally combined.

Q5. Why is load testing required?

Models rest on assumptions. Load tests validate them and expose bottlenecks such as database locks or connection limits.

Q6. Sticky sessions and the better alternative?

Sticky sessions pin a user to one server because session data is in its memory. Better: stateless servers with sessions in Redis or JWT, so any node can serve any request.

Q7. How does redundancy improve availability?

Two independent 99% servers in parallel: $1 - (0.01 \times 0.01) = 99.99\%$. Shared parts (balancer, database) still limit the real figure.

5. Scenario Questions

Q8. Your site crashes at every sale start; app CPU is 100% but the database is at 30%.

The app tier is the bottleneck. Scale out app nodes behind the balancer, enable auto-scaling and pre-warm before sales, cache product pages and use a CDN. Load test at 1.5x expected peak before each campaign.

Q9. One node of a three-node cluster dies at 2 a.m. What should happen?

Health check fails, the balancer stops routing to it, the two survivors take the load and an alert fires. Each survivor now carries 50% more, so if they ran above about 66% before, they are overloaded. That is why N+1 matters.

Q10. Users are randomly logged out after you added a second app server.

Sessions live in each server's memory and round robin sends users to the other server. Quick fix: sticky sessions. Proper fix: shared session store (Redis) or JWT.

Q11. Cut the cloud bill: traffic peaks 8-11 pm and servers idle at 15% otherwise.

Use scheduled and metric-based auto-scaling, reserved capacity for the baseline only, and spot or on-demand for peaks. Keep a minimum of 2 nodes and check scale-out is fast enough.

Q12. Admission portal: 300,000 applicants over 3 days, 40 requests each, peak hour is 30% of a day. One server does 150 RPS at 70% target.

$100,000/\text{day} \times 40 = 4\text{M}$ requests; $30\% = 1.2\text{M}$ in $3,600\text{ s} = \text{about } 333\text{ RPS}$. Usable per server 105 RPS, so $333 / 105 = 3.2$, so 4 servers + 1 spare = **5 servers**. Test a 2-3x higher last-day rush.